

M-DAFTO: Multi-Stage Deferred Acceptance Based Fair Task Offloading in IoT-Fog Systems

Chittaranjan Swain , Graduate Student Member, IEEE, Manmath Narayan Sahoo , Member, IEEE,
Anurag Satpathy , Graduate Student Member, IEEE, Sambit Bakshi , Senior Member, IEEE,
and Soumya K. Ghosh , Senior Member, IEEE

Abstract—Resource-constrained Internet of Things (IoT) devices depend on remote Cloud/Fog Nodes (FNs) to execute deadline-sensitive services. Offloading computations of real-time services to a remote cloud server results in intolerable latency due to intermittent channels, higher transmission delays, and scarce spectrum resources. Therefore, offloading to nearby FN is preferable; however, it introduces several significant issues: (i) allocation of limited FN resources, (ii) deadline constraint of heterogeneous services, and (iii) requirement of computationally inexpensive and scalable strategies. This article proposes a M-DAFTO model to tackle the abovementioned issues and generate a fair offloading plan in polynomial time. The offloading problem is modeled as a many-to-one matching game with maximum and minimum quotas at each FN. Because the deferred acceptance (DA) algorithm fails to operate with minimum quotas, we adopt a variant of the DA algorithm, a multistage deferred acceptance (MSDA) algorithm, to solve the offloading problem. The overall goal of M-DAFTO is to reduce the aggregate offloading delay with increased assignment of tasks to FNs. Extensive simulation and analysis confirm a 30.26% and a 93.53% reduction in offloading delay and outages (unassigned tasks) compared to the baselines.

Index Terms—Task offloading, IoT, fog systems, matching theory, deferred acceptance algorithm, analytic hierarchy process, technique for order performance by similarity to ideal solution.

I. INTRODUCTION

THE rapid growth of the Internet of Things (IoT) has facilitated the emergence of real-time services such as online gaming, video streaming, facial recognition, health care, virtual reality (VR), augmented reality (AR), and automated driving [1], [2], [3]. The recent statistics published by the International Data Corporation (IDC) state that 55.7 billion devices will execute such real-time services by 2025, 75% of which will be IoT. These IoT devices are expected to generate 73.1 ZB of data [4].

Received 22 February 2023; revised 6 June 2024; accepted 26 July 2024.
Date of publication 2 August 2024; date of current version 30 December 2024.
(Corresponding author: Manmath Narayan Sahoo.)

Chittaranjan Swain, Manmath Narayan Sahoo, and Sambit Bakshi are with the Department of Computer Science and Engineering, National Institute of Technology, Rourkela, Odisha 769008, India (e-mail: chittaranjanswain518@gmail.com; sahoom@nitrrkl.ac.in; bakshisambit@ieee.org).

Anurag Satpathy is with the Department of Computer Science, Missouri University of Science and Technology, Rolla, MO 65409 USA (e-mail: anurag.satpathy@mst.edu).

Soumya K. Ghosh is with the Department of Computer Science and Engineering, Indian Institute of Technology, Kharagpur, West Bengal 721302, India (e-mail: skg@iitkgp.ac.in).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TSC.2024.3436648>, provided by the authors.

Digital Object Identifier 10.1109/TSC.2024.3436648

Real-time services often require adequate computing resources for processing such sizeable data. However, IoT devices are often resource-constrained and incapable of performing large-scale computations, relying on remote devices for processing [5].

Remote processing, often referred to as *offloading*, is performed in two modes depending on where the services are executed. The first execution mode directly offloads the services to a remote cloud server [6]. This mode of execution is infeasible for real-time services as higher latency is experienced due to *physical distance*, *intermittent WAN latency*, *network congestion*, and *scarce spectrum resources* [5]. To overcome the pitfalls, an alternate execution mode involves offloading computations to nearby fog nodes (FNs) instead of the remote cloud server [1]. Additionally, offloading to FNs provides real-time mobility support, location awareness, and geo-distributed services [7]. Although offloading services to FNs is a sensible option, generating an offloading plan for a densely connected IoT-Fog network with heterogeneous resources is non-trivial and introduces multiple research challenges. First, the delay guarantee of real-time services is difficult to achieve, especially with the stochastic computational demands of IoTs and FNs. Second, the contention of limited FN resources in a massive IoT network often complicates the offloading decisions. Finally, the requirement of *scalable*, *computationally-inexpensive*, and *non-clustered* assignment assists in effectively utilizing the limited FN resources and avoiding congestion.

The existing works on offloading focused on reducing the offloading delay or outages, i.e., services overshooting the prescribed deadlines. In this regard, works in [4], [8], [9] adopted meta-heuristic strategies to generate an offloading schedule. Though these approaches are acceptable, they are *compute-intensive*, *not scalable*, and *non-polynomial*. Moreover, the above-discussed approaches ignore the deadlines associated with real-time services, thereby leading to quality-of-service (QoS) violations [1]. The strategies discussed in [3], [10], [11], [12] aimed to optimize the offloading delay adhering to the QoS requirements in terms of the deadline. However, they may suffer from clustered assignments, resulting in ineffective utilization and congestion at the FNs. Therefore, this work proposes a framework *M-DAFTO* to simultaneously reduce *offloading delays* and *outages*. Adopting a multistage deferred acceptance (MSDA) algorithm with maximum and minimum quotas at the FNs assists in generating a polynomial time non-clustered assignment.

A. Contributions

- We propose a novel framework *M-DAFTO* that generates a fair offloading schedule focused on reducing the offloading delay without compromising the QoS of services expressed as their deadlines.
- The offloading between IoTs and FNs is designed as a many-to-one matching game considering maximum and minimum quotas at each FN. The adoption of such a solution approach introduces two challenges. First, setting the minimum quota is arduous due to resource heterogeneity at FNs. Second, generating the preferences of the FNs is complex, especially considering multiple criteria such as offloading delay, QoS, and feasible execution options. We devise a novel strategy in Section V-A to assign the minimum quotas at FNs, which addresses the first concern. We also introduce a term called *urgency* in Section V-B that captures the preferences of services obtained using a combination of analytic hierarchy process (AHP) and technique for order performance by similarity to the ideal solution (TOPSIS).
- To obtain a *fair* offloading schedule, we adopt a variant of the deferred acceptance algorithm called MSDA that takes as input the processing order of the services in the form of a precedence list (PL). The PL construction, as presented in Section V-C, considers the demands, QoS, and feasible execution options of different services.
- The performance of *M-DAFTO* is compared against three different baselines: a matching theory framework for tasks offloading in fog computing for IoT systems (MBTO) [13], heuristic-based task offloading strategy [12], greedy placement method (GPM) [14], and efficient load-balanced strategy for task offloading in IoT-Fog systems (LETO) [15]. Through exhaustive simulation and analysis, we infer that *M-DAFTO* achieves a 30.26% reduction in offloading delay and 93.53% reduction in the number of outages compared to the baselines.

To the best of our knowledge, this is the first work that aims to reduce the offloading delay and outages using a maximum-minimum quota-based matching game. A preliminary version of this work is published in a conference [15], where we discussed a basic framework for modeling the offloading as a many-to-one matching game with a maximum-minimum quota at the FNs. The presented work in this article is a significant extension that overcomes the drawbacks and provides a comprehensive view of the solution strategy with extensive simulation and analysis.

The rest of the paper is organized as follows. Section II discusses the literature we reviewed. Section III presents the system model in detail. The overall problem is formulated in Section IV. Sections IV-C and V discuss the offloading game and the solution approach. Performance analysis of *M-DAFTO* is presented in Section VI, and conclusions are drawn in Section VII.

II. RELATED WORKS

This section discusses the relevant literature on offloading from two aspects. First, we discuss strategies that solely focus on addressing the reduction in the *offloading delays*. Reducing

the *offloading delay* without any impetus on the maximum tolerable latency of individual services may not be suitable for real-world services such as AR and VR, with stringent QoS expressed in terms of their deadlines [16]. For such applications, QoS violations can result in user dissatisfaction and loss of revenue for the service providers (SPs) in the long run, [15]. Therefore, the second part of the survey discusses strategies that consider both the *offloading delay* and *outages* while generating an offloading plan.

The works in [4], [8], [9] developed meta-heuristic-based solutions to reduce the *offloading delay* in complex fog networks. In this regard, Kishor et al. [4] discussed an efficient offloading policy based on a smart ant colony optimization (SACO). Alternatively, a genetic algorithm (GA)-based swift offloading policy for sensors in an intelligent city network is discussed in [8]. On the other hand, Wang and Chen [9] considered a setup that considered both local and FN resources in making joint offloading decisions. A GA and simulated annealing (SA)-based hybrid strategy is adopted to generate the offloading schedule. Some other works discussed in [17], [18], [22], and [23] have also modeled offloading as an optimization. Although the above-discussed approaches generate acceptable solutions, they are *inefficient*, *time-consuming*, and *not scalable* [1]. To overcome the lacunas of optimization solvers, authors in [13], [19], [20], [21] proposed game-theoretic solutions. In this regard, Nowak et al. [21] modeled the association between the access point and cloudlets as a non-cooperative game. On the other hand, Chitti et al. [13], and Ali et al. [20] formulated the offloading as a one-to-many matching game and solved it using a variation of the DA algorithm. Alternatively, authors in [19] imposed maximum and minimum quotas at the FNs to achieve a non-clustered assignment. They solved the matching problem using a multi-stage deferred acceptance (MSDA) algorithm. Additionally, works in [24], [25] focused on reducing the *offloading delay* in different environments.

As discussed earlier, a more realistic setup involves running services within their specified deadlines. In this context, Zakariya et al. [10] discussed a GA-based solution that aims to minimize the outages without compromising much with the *offloading latency*. Deng et al. [11] proposed a novel hybrid meta-heuristic scheme to execute maximum offloaded tasks within their respective deadlines to obtain the best of both worlds. The authors hybridized GA and Particle swarm optimization (PSO) to obtain a sub-optimal solution. Deviating from meta-heuristic-based solutions, Adhikari et al. [3] proposed a delay-dependent priority-aware heuristic offloading strategy called DPTO. The authors assign task priorities based on their deadlines and use a multilevel feedback queue to schedule offloaded tasks. Alternatively, in [15], the authors formulated the offloading as a one-to-many matching-based with maximum and minimum quotas at the FNs to achieve a non-clustered assignment. The MSDA algorithm was adopted to solve the matching game, which yielded a PL fair offloading schedule. Some other works on vehicular networks focused on *offloading latency* reduction with deadline specifications are discussed in [26], [27], [28].

It may be noted from the above discussion that offloading has been studied and investigated in the existing literature.

TABLE I
COMPARISON OF OFFLOADING SCHEMES

Work	Heuristic-Based	Evolutionary-Based	Offloading Delay Centric	Outage Centric
J. He [17] (2022)	✗	✓	✓	✓
Swain et al. [15] (2021)	✓	✗	✓	✓
Zakariya et al. [10] (2021)	✗	✓	✓	✓
Kishor and Chakarbarty [4] (2021)	✗	✓	✓	✗
Li and Zhu [18] (2020)	✗	✓	✓	✗
Bao and Liu [12] (2019)	✓	✗	✓	✗
Abouaomar and Kobbane [19] (2018)	✓	✗	✓	✗
Chitti et al. [13] (2018)	✓	✗	✓	✗
Ali et al. [20] (2018)	✓	✗	✓	✗
Nowak et al. [21] (2018)	✓	✗	✓	✗
M-DAFTO (Proposed)	✓	✗	✓	✓

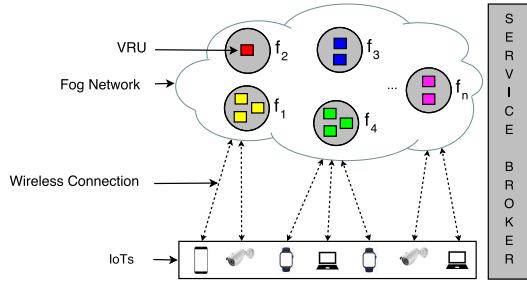


Fig. 1. An IoT-Fog network comprising multiple IoT devices and FNs communicating over a wireless channel.

Most of the literature focuses on reducing the *offloading delay* without the impetus to meet the deadlines. Moreover, the limited literature considers both optimization and meta-heuristics-based solutions. The former is *time-consuming* and suffers from *inefficiency*, whereas the latter is *computationally expensive* and is *not scalable*. Therefore, this paper proposes a matching-based solution approach called *M-DAFTO*. It simultaneously aims to minimize the *offloading delay* and *outages* in a polynomial time, ensuring a non-clustered assignment at the FNs. This can be achieved by introducing a minimum quota at each FN. Further, Section-V-A discusses a minimum quota determination algorithm. Table I provides a comparative analysis of literature based on solution approaches and parameters used.

III. SYSTEM MODEL

Fig. 1 represents the overall architecture of an IoT-Fog network comprising m IoTs and n FNs. Each IoT d_i generates an indivisible task t_i and $\mathcal{T} = \{t_1, t_2, t_3, \dots, t_m\}$ captures all such tasks. The device d_i , i.e., $i \in [1, m]$, is assumed to be resource-constrained and incapable of local execution [1]. Hence, for execution, the devices are computationally dependent

on nearby FNs captured by $\mathcal{F} = \{f_1, f_2, f_3, \dots, f_n\}$. Moreover, an offloading request of t_i is represented as a quadruple $\langle s_i^{in}, \eta_i, \Delta_i, s_i^{out} \rangle$. Here, s_i^{in} and s_i^{out} denote input and output size (in *bits*) whereas η_i and Δ_i represent the computational demand (in *cycles*) and deadline (in *s*). The computational resource of a FN $\mathbb{C}_j$ (*cycles/sec*) is logically partitioned into ' x ' isolated executable entities termed virtual resource units (VRUs) [1], such that $x \in \mathbb{Z}^+$. Let λ_j (*cycles/sec*) be the compute capability of a VRU at f_j , derived by dividing FN's computational capability by the number of VRUs. We assume computationally identical VRUs within a FN [1], [5]. For instance, if f_j has a computational capability of 4 *GHz*, and $x = 4$, then each VRU's assigned computational capacity would be 1 *GHz*. Heterogeneity is imparted by ensuring $\lambda_j \neq \lambda_{j'}$, $j \neq j'$. Opting for homogeneous VRUs at a FN simplifies the modeling and problem formulation [5]. Regardless of the differences in computational capability among VRUs within and across FNs, *M-DAFTO* effortlessly generates the offloading plan. The number of VRUs at f_j is denoted as its maximum quota $h(f_j)$, representing the number of offloaded tasks simultaneously executed at f_j . We also impose a minimum quota $l(f_j)$ at f_j to guarantee a non-clustered assignment (refer to Section I).

Relationship Between FN Parameters: Considering the compute capacity of a VRU λ_j and that of FN f_j , $\lambda_j \leq \mathbb{C}_j$ holds such that $\mathbb{C}_j = x * \lambda_j$, $x \in \mathbb{Z}^+$. Furthermore, if f_j has ' x ' VRUs, its maximum quota can be set as $h(f_j) = x$. Note that our model does not focus on determining ' x '; we assume it to be an integral value.

Delays in Offloading: The latency incurred in the offloading procedure is the aggregation of the following delays (i) *communication delay* and (ii) *execution delay*. The former involves transmitting the offloaded task to the designated FN and receiving the processed output. The latter captures the delay in executing the offloaded task at the FN.

A. Communication Delay

The aggregate delay in the communication phase encompasses (i) *transmission delay* and (ii) *receiving delay*.

Transmission Delay: It captures the time needed to send a task t_i from a device d_i to FN f_j for its computation. Following [1], we assume the presence of an active uplink and downlink channel for each IoT device captured as $B_{i,j}^u$ and $B_{i,j}^d$, respectively. Let p_i^{tp} represent the transmission power of IoT device d_i , and $h_{i,j}$ indicates the channel gain from d_i to f_j with noise power n_0 . Therefore, the effective uplink data rate $\mathbb{R}_{i,j}$ is derived as per (1).

$$\mathbb{R}_{i,j} = B_{i,j}^u * \log_2 \left(1 + \frac{p_i^{tp} * h_{i,j}}{n_0} \right) \quad (1)$$

Taking s_i^{in} and $\mathbb{R}_{i,j}$, the transmission delay ($\mathcal{D}_{i,j}^{td}$) from IoT device d_i to FN f_j can be calculated as per (2).

$$\mathcal{D}_{i,j}^{td} = \frac{s_i^{in}}{\mathbb{R}_{i,j}} \quad (2)$$

Receiving Delay: It indicates the time to receive the processed output from f_j . Let p_j^{tp} denote the transmission power of f_j , and $h_{j,i}$ be the channel gain between f_j and d_i . The downlink rate $\mathcal{R}_{j,i}$ is determined as per (3).

$$\mathcal{R}_{j,i} = B_{j,i}^d * \log_2 \left(1 + \frac{p_j^{tp} * h_{j,i}}{n_0} \right) \quad (3)$$

Considering s_i^{out} and $\mathcal{R}_{j,i}$, the receiving delay ($\mathcal{D}_{j,i}^{rd}$) is derived as (4).

$$\mathcal{D}_{j,i}^{rd} = \frac{s_i^{out}}{\mathcal{R}_{j,i}} \quad (4)$$

B. Computation of Execution Delay

The execution delay ($\mathcal{D}_{i,j}^{ed}$) of task t_i at FN f_j depends on (i) computational requirement of t_i , i.e., η_i and (ii) computational capability of a VRU at f_j , i.e., λ_j . Considering these parameters, it can be derived as per (5).

$$\mathcal{D}_{i,j}^{ed} = \frac{\eta_i}{\lambda_j} \quad (5)$$

C. Offloading Delay

The offloading delay ($\mathcal{D}_{i,j}^{off}$) of t_i is the sum of all delays incurred while offloading (transmission, execution, and receiving) and can be derived as per (6).

$$\mathcal{D}_{i,j}^{off} = \mathcal{D}_{i,j}^{td} + \mathcal{D}_{i,j}^{ed} + \mathcal{D}_{j,i}^{rd} \quad (6)$$

IV. PROBLEM FORMULATION

In this work, we formulate the overall problem as a minimization function to reduce the aggregate offloading delay. We also ensure a non-clustered assignment of tasks to FNs without compromising the objective mentioned above.

A. Constraints on Offloading

Before defining the constraints imposed, we introduce a binary indicator variable $\chi_{i,j}$ expressed as per (7).

$$\chi_{i,j} = \begin{cases} 1 & \text{If task } t_i \text{ is assigned to the FN } f_j \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

1) *Delay Constraint:* This constraint guarantees that the offloaded tasks meet their deadline requirements, which is crucial for time-sensitive tasks. Equation (8) captures this constraint.

$$\mathcal{D}_{i,j}^{off} \leq \Delta_i \quad (8)$$

2) *Assignment Constraint:* It ensures that a task can be offloaded at most one FN $f_j \in \mathcal{F}$. Further, this constraint captures a full offloading scenario considered in *M-DAFTO* and is expressed as (9).

$$\sum_{j=1}^n \chi_{i,j} \leq 1; \quad \forall i \in [1, m] \quad (9)$$

3) *Capacity Constraint:* A maximum of $h(f_j)$ offloaded tasks can be executed parallelly at a FN $f_j \in \mathcal{F}$. This is captured

in (10).

$$\sum_{i=1}^m \chi_{i,j} \leq h(f_j); \quad h(f_j) > 0; \quad \forall j \in [1, n] \quad (10)$$

4) *Minimum Quota Constraint:* Minimum quota $l(f_j)$ of a FN $f_j \in \mathcal{F}$ is upper-bounded by its maximum quota $h(f_j)$, as captured in (11).

$$l(f_j) \leq h(f_j); \quad l(f_j) \geq 0; \quad \forall j \in [1, n] \quad (11)$$

5) *Max-Min Constraint:* The max-min constraint ensures that the offloaded tasks, i.e., $|\mathcal{T}|$ should be within the range of the cumulative minimum quotas and the sum of maximum quotas, considering all $f_j \in \mathcal{F}$. This requirement is expressed as (12).

$$\sum_{j=1}^n l(f_j) \leq |\mathcal{T}| \leq \sum_{j=1}^n h(f_j) \quad (12)$$

B. Objective of M-DAFTO

M-DAFTO aims to reduce the total offloading delay in the IoT-Fog network.

1) *Minimization of Total Offloading Delay:* The total offloading delay ($\mathcal{A}^{off}$) incurred in successfully offloading the tasks to FNs in the IoT-Fog network and can be derived as per (13).

$$\mathcal{A}^{off} = \left(\sum_{i=1}^m \sum_{j=1}^n \chi_{i,j} * \mathcal{D}_{i,j}^{off} \right) \quad (13)$$

2) *Objective Formulation and Constraints for M-DAFTO:* *M-DAFTO*'s objective is expressed as per (14a). Conditions (14b), (14c), (14d), (14e), and (14f) respectively capture the delay, assignment, capacity, minimum quota, and max-min constraints. The values of variables are captured as Constraint (14g).

$$\text{minimize} : \mathcal{A}^{off} \quad (14a)$$

$$\text{s.t.} \quad \mathcal{D}_{i,j}^{off} \leq \Delta_i \quad (14b)$$

$$\sum_{j=1}^n \chi_{i,j} \leq 1 \quad (14c)$$

$$\sum_{i=1}^m \chi_{i,j} \leq h(f_j); \quad h(f_j) > 0 \quad (14d)$$

$$l(f_j) \leq h(f_j); \quad l(f_j) \geq 0, \quad (14e)$$

$$\sum_{j=1}^n l(f_j) \leq |\mathcal{T}| \leq \sum_{j=1}^n h(f_j) \quad (14f)$$

$$\forall i \in [1, m], \quad \forall j \in [1, n] \quad (14g)$$

The problem captured as (14a) is $\mathcal{NP}$ -Hard enforcing an ardent challenge is obtaining a polynomial time solution [1]. In this regard, *M-DAFTO* models the offloading as a matching game focused on achieving a fair assignment in polynomial time.

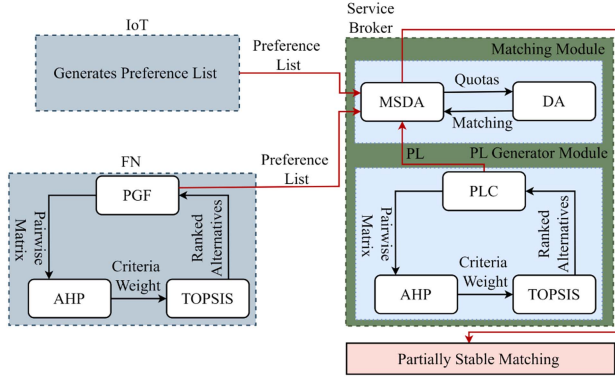


Fig. 2. Components of *M-DAFTO* and their interactions.

The working of *M-DAFTO* is shown in Fig. 2, and the problem is subsequently formulated.

C. Task Offloading as a Matching Game

The objective of *M-DAFTO* is to minimize total offloading delay. *M-DAFTO* achieves this objective through a matching game between tasks and FNs, considering the preferences of both agents.

Relationship of Task-FN Matching With *M-DAFTO*'s Objective: *M-DAFTO* considers a many-to-one matching between tasks ($\mathcal{T}$) and FNs ($\mathcal{F}$), referred to as agents. Before assignment (Algorithm 9), an agent ranks some/all agents in the opposite set (Algorithm 2). For instance, tasks rank the FNs in increasing order of offloading delays. FNs order tasks in decreasing urgencies (17), which considers the deadline, offloading delay, and the number of feasible execution execution options. The proposal order is derived as a PL by considering the CPU demand, the deadline, and the number of feasible execution options for each task (Algorithm 8). These data structures serve as inputs to the mapping and facilitate achieving the objective outlined in (14a). The offloading delay and outages significantly impact the mapping game by directly influencing the preferences of tasks and FNs. For instance, a task would ideally prefer to be offloaded to an FN with a lower offloading delay, making it logical to rank FNs in increasing order of offloading delays. Conversely, FNs prefer tasks with higher urgency to meet their delay requirements.

Definition 1. (Preference Relation): A matching over set $\mathcal{T}$ and $\mathcal{F}$ has the same number of preference relations, i.e., $\succ_{t_i}$ and $\succ_{f_j}$. The former enables an agent $t_i \in \mathcal{T}$ to rank the agents in $\mathcal{F}$, whereas the latter allows conversely.

A preference relation $t_i \succ_{f_j} t_{i'}$ indicates that f_j assigns a higher rank to t_i over $t_{i'}$. In *M-DAFTO*, an agent's preference is *strict*, *transitive*, and *incomplete* relation. Collectively, the preference relations of an agent $a \in \mathcal{T} \cup \mathcal{F}$ is called its *preference profile* and is captured as $pref(a)$.

Definition 2. (Offloading Game): The matching game between $\mathcal{T}$ and $\mathcal{F}$ is a many-to-one mapping $\Upsilon: \mathcal{T} \cup \mathcal{F} \rightarrow 2^{\mathcal{T}} \cup 2^{\mathcal{F}}$, satisfying the conditions below:

Condition 2.1: Each $t_i \in \mathcal{T}$, $|\Upsilon(t_i)| \neq 1$ and $\Upsilon(t_i) \subseteq \mathcal{F}$

Condition 2.2: Each $f_j \in \mathcal{F}$, $\Upsilon(f_j) \subseteq \mathcal{T}$, and $|\Upsilon(f_j)| \leq h(f_j)$

Condition 2.3: For t_i and f_j , $t_i \in \Upsilon(f_j) \iff f_j \in \Upsilon(t_i)$

Condition (2.1) specifies that each task t_i can be assigned to only one FN, signifying a full offloading scenario. Considering the capacity of FNs to handle multiple tasks, Condition (2.2) ensures that FN f_j is constrained to hosting a maximum of $h(f_j)$ offloaded tasks. Finally, Condition (2.3) emphasizes the symmetry in the task-to-FN mapping, affirming that t_i is paired with f_j if and only if f_j is paired with t_i in Υ .

Definition 3. (Individual Rationality): The assignment Υ is *individually rational* iff it satisfies the following:

Condition 3.1: $t_i \in \Upsilon(f_j)$ and $f_j \notin pref(t_i)$.

Condition 3.2: $f_j \in \Upsilon(t_i)$ and $t_i \notin pref(f_j)$.

Individually rational outcomes, as delineated in Conditions (3.1) and (3.2) of Definition 3, pertain to assignments or allocations that take into account acceptable participants or agents [29]. This concept centers on meeting the criteria for individual satisfaction by accommodating the decision-making preferences of each participant. Additionally, in the context of matching-based assignments, it is imperative to uphold each agent's rationality, ensuring both rational and efficient assignments.

Definition 4. (Blocking Pair): A pair (t_i, f_j) blocks the matching Υ subject to the adherence of the following conditions:

Condition 4.1: $t_i \notin \Upsilon(f_j)$ and $f_j \notin \Upsilon(t_i)$

Condition 4.2: $f_j \succ_{t_i} f_{j'}$, $f_{j'} \in \Upsilon(t_i)$ and $|\Upsilon(f_j)| = 0$

Condition 4.3: $t_i \succ_{f_j} t_{i'}$, where $t_{i'} \in \Upsilon(f_j)$

Condition (4.1) states that t_i and f_j are not matched in Υ . Condition (4.2) reflects that t_i blocks the matching Υ iff it prefers f_j over its current match $f_{j'}$. According to Condition (4.3), f_j blocks the matching Υ , if there exists a $t_{i'} \in \Upsilon(f_j)$ such that it prefers t_i over $t_{i'}$.

Definition 5. (Fairness): A fair matching Υ has no *blocking* pairs.

The absence of blocking pairs is the crucial criterion to achieve *fairness*. However, obtaining a fair match meeting the minimum quota constraint expressed in (11) is not always possible. This can be illustrated using an example scenario in Appendix A.3, available online. Therefore, a relaxed definition of *fairness* called *PL-fairness* is proposed that disallows the presence of PL-blocked pairs [30].

Definition 6. (PL-Blocked Pair): A matching Υ is said to be PL-blocked by a pair of agents (t_i, f_j) iff it satisfies the conditions of Definition 4 and the following condition

$$t_i \succ_{PL} t_{i'}, i \neq i' \quad (15)$$

Equation (15) states that t_i appears before $t_{i'}$ in the PL. Moreover, the PL enforces a strict processing order for the tasks to ensure a *PL-fair* matching.

Definition 7. (PL-Fairness): If a matching Υ contains no *PL-Blocked Pair*, it is considered a *PL-fair* matching.

A PL holds significance in establishing the priority or preferences of agents during each stage of the matching process. Its crucial role lies in shaping the outcomes of the matching algorithm by providing a predefined order in which agents are considered for potential matches. This mechanism promotes

fairness and contributes to the stability and consistency of matching process outputs.

Definition 8. (Wastefulness): A wasteful matching Υ satisfies the following conditions:

Condition 8.1: $\exists f_j$ such that $|\Upsilon(f_j)| < l(f_j)$

or

Condition 8.2: $\exists f_j$ such that:

a) $\exists (t_i, f_{j'}) \mid t_i \in \Upsilon(f_{j'})$ and $f_{j'} \in \Upsilon(t_i)$

b) $f_j \succ_{t_i} f_{j'}$ and $|\Upsilon(f_j)| < h(f_j)$

c) There exists another matching Υ' such that $\Upsilon' = (\Upsilon \cup \{(t_i, f_j)\}) \setminus \{(t_i, f_{j'})\}$ and $|\Upsilon(f_{j'})| \geq l(f_{j'})$

Condition (8.1) implies that there exists FN $f_j \in \mathcal{F}$, the minimum quota of which is not satisfied. Condition (8.2)(a) ensures that t_i and f_j are matched in Υ . Condition (8.2)(b) affirms that a task t_i can claim an empty VRU at f_j if the task t_i prefers f_j over its current match $f_{j'}$ and f_j can accommodate it. Condition (8.2)(c) implies that there exists another matching Υ' where (t_i, f_j) can be paired without violation of minimum quota constraint at $f_{j'}$ in Υ . The concept of the “wasteful property” elucidates a scenario where specific agents are allocated less preferred assignments in later stages, even though they had the chance to secure more preferred options earlier. This is deemed as inefficiency or “waste” within the allocation process.

Definition 9. (Strong Stability): A strongly stable matching Υ satisfies properties of *individual rationality*, *fairness*, and *non-wastefulness*.

With maximum and minimum quotas at each FN, the *strongly stable* matching is not always possible [30]. The details can be found in Appendices A.1, A.2, and A.3, available online.

Definition 10. (Partial Stability): A partially stable matching Υ satisfies the properties of *individual rationality* and *PL-fairness*.

V. SOLUTION APPROACH

Traditionally, the deferred acceptance (DA) algorithm has been successfully used to obtain a *stable* assignment in different scenarios [1], [20], [31]. However, the DA algorithm fails to generate a matching that meets the minimum quota of agents [30]. This is highlighted using an example presented in Appendix B, available online. Therefore, in *M-DAFTO*, we utilize an improved version of the DA algorithm called the MSDA algorithm that achieves a partially stable assignment. Although the adoption of the MSDA algorithm meets the design objective of *M-DAFTO*, it introduces the following additional research challenges. *First*, in *M-DAFTO* context, setting the minimum quota is non-trivial owing to the heterogeneous nature of FNs. *Second*, designing an agent’s preferences is particularly challenging as *M-DAFTO* caters to multiple parameters. *Third*, determining the order of the tasks for processing is crucial and more challenging because of their heterogeneous offloading requests. Subsequently, we discuss sophisticated procedures to address each research challenge.

A. Determination of Minimum Quota of FNs

In [15], [19], the authors have considered a random assignment policy for setting the minimum quota of FNs in the range

Algorithm 1: Minimum Quota Determination (MQD):

Input: $\mathcal{T}, \mathcal{F}; h(f_j), \lambda_j, \forall j \in [1, n]$

Result: $l(f_j), \forall f_j \in \mathcal{F}$

```

1 Initialize:  $l(f_j) = 0, \forall f_j \in \mathcal{F}$ 
2  $[\lambda^{max}, f_j] = \max_{f_j \in \mathcal{F}} (\lambda_j)$ 
3  $l^{max} = \min((h(f_j), \lfloor \frac{|\mathcal{T}|}{|\mathcal{F}|} \rfloor))$ 
4  $l(f_j) = l^{max}$ 
5 for each  $f_{j'} \in \mathcal{F} \setminus \{f_j\}$  do
6    $l' = \left( \frac{\lambda_{j'}}{\lambda^{max}} \right) \cdot l^{max}$ 
7    $l(f_{j'}) = \min(l', h(f_{j'}))$ 
```

$[0, \lfloor \frac{|\mathcal{T}|}{|\mathcal{F}|} \rfloor]$. Here, $|\mathcal{T}|$ and $|\mathcal{F}|$ represent the total number of tasks to be offloaded and the number of FNs in the network. Similar assignment policies have also been followed in [30], [32]. However, such assignment policies are irrational and cannot directly be incorporated in *M-DAFTO* for the following reasons. For a pair of FNs, $(f_j, f_{j'}) \in \mathcal{F}$ with $\lambda_j > \lambda_{j'}$, a random allocation can lead to an assignment such that $l(f_{j'}) > l(f_j)$, which is *inconsistent*. This *inconsistency* compromises the *offloading* delay by assigning a higher minimum quota to a less computationally efficient FN. This results in undesirable allocations and impacts *M-DAFTO* design objective. Therefore, a sophisticated minimum quota determination (MQD) procedure is proposed, which takes the computational capability $\lambda_j, \forall f_j \in \mathcal{F}$ and ensures *consistent* assignments.

1) *Minimum Quota Determination:* The overall working of MQD follows Algorithm 1. (i) two sets of agents - $\mathcal{T}, \mathcal{F}$, (ii) maximum quota $h(f_j), \forall f_j \in \mathcal{F}$, and (iii) computational capability $\lambda_j, \forall f_j \in \mathcal{F}$ are taken as input. The algorithm outputs the minimum quota $l(f_j), \forall f_j \in \mathcal{F}$. MQD executes as follows. All FNs’ minimum quotas are initialized to 0, i.e., $l(f_j) = 0, \forall f_j \in \mathcal{F}$. Let f_j be the most computationally efficient FN, and its computational capability is λ^{max} . The minimum quota of f_j is then set to $\min((h(f_j), \lfloor \frac{|\mathcal{T}|}{|\mathcal{F}|} \rfloor))$. Allocating minimum quota to the FNs in the range $[0, \lfloor \frac{|\mathcal{T}|}{|\mathcal{F}|} \rfloor]$ ensures the feasibility of allocation (refer to (14f)). we adopt a proportionate assignment of minimum quotas for remaining FN $(f_{j'} \in \mathcal{F} \setminus \{f_j\})$ in decreasing order of their computational capabilities. This ensures for a pair of FNs $(f_j, f_{j'}) \in \mathcal{F}$ if $\lambda_j > \lambda_{j'}$ then $l(f_j) > l(f_{j'})$, improving the objective of *M-DAFTO*.

B. Preference Profiles of Agents

The preference of an agent $a \in \mathcal{T} \cup \mathcal{F}$ is captured as $pref(a)$. It’s important to note that an agent’s preference reflects its level of flexibility, representing the number of potential partners an agent can choose. In the context of *M-DAFTO*, the degree of freedom for a task t_i and FN f_j is precisely defined by $|pref(t_i)|$ and $|pref(f_j)|$, respectively.

1) *Preference of Tasks:* Each task $t_i \in \mathcal{T}$ ranks the FNs in non-decreasing order of the *offloading delay* where $\forall f_j \in \mathcal{F}$ and $\Delta_i \geq \mathcal{D}_{i,j}^{off}$, thereby boosting the objective of *M-DAFTO*. For instance, a task t_i prefers a FN f_j over $f_{j'}$ iff the *offloading*

Algorithm 2: Preference Generation of FN (PGF).

Input: $\mathcal{C}, \mathcal{T}, \mathcal{F}$
Result: $pref(f_j), \forall f_j \in \mathcal{F}$
1 **Initialize:** $pref(f_j) = \Phi, \forall f_j \in \mathcal{F}$
2 **for each** $f_j \in \mathcal{F}$ **do**
3 $\mathcal{T}' = \{t_i | \Delta_i \geq \mathcal{D}_{i,j}^{off}\}$
4 **if** $|\mathcal{T}'| == 1$ **then**
5 $pref(f_j) = t_i$
6 **else if** $|\mathcal{T}'| > 1$ **then**
7 $\mathcal{W} = \text{ACW}(\mathcal{C})$
8 **for each** $t_i \in \mathcal{T}'$ **do**
9 **for each** $c_x \in \mathcal{C}$ **do**
10 **if** $x==1$ **then**
11 $\mathbb{D}_{f_j}[t_i][c_x] = \Delta_i - \mathcal{D}_{i,j}^{off}$
12 **if** $x==2$ **then**
13 $\mathbb{D}_{f_j}[t_i][c_x] = |\mathcal{T}'|$
14 $pref(f_j) = \text{TRD}(\mathbb{D}_{f_j}, \mathcal{W}, \mathcal{C}, \mathcal{T}')$
15 **else**
16 Continue

delay at f_j is less than $f_{j'}$. Therefore,

$$f_j \succ_{t_i} f_{j'} \iff \mathcal{D}_{i,j}^{off} < \mathcal{D}_{i,j'}^{off}; j \neq j' \quad (16)$$

2) *Task Urgency Computation:* FNs focus on executing tasks within their prescribed deadlines to meet *M-DAFTO*'s agenda. However, due to incomplete preferences, preferential offloading based on the nearest deadline may not guarantee allocations. Therefore, a metric called *urgency* associated with each task $t_i \in \mathcal{T}$ is coined. It captures the priority of a task t_i at FN f_j considering (i.) buffer time: $\Delta_i - \mathcal{D}_{i,j}^{off}$ and (ii.) number of feasible executing options: $|pref(t_i)|$. The former assists in prioritizing tasks with offloading delay closer to the deadline, whereas the latter helps reduce unassigned tasks. Therefore, a merged valuation helps achieve *M-DAFTO*'s objective. The urgency of t_i at f_j ($u_{t_i}^j$) is derived as per (17). The weights (w_1, w_2) and task urgencies are determined via Algorithms 3 and 7, respectively.

$$u_{t_i}^j = w_1 * \frac{1}{(\Delta_i - \mathcal{D}_{i,j}^{off})} + w_2 * \frac{1}{|pref(t_i)|} \quad (17)$$

3) *Preference of FNs:* A pair of tasks t_i and $t_{i'}$ to be executed at f_j are ranked in non-increasing order of their urgencies. Therefore,

$$t_i \succ_{f_j} t_{i'} \iff u_{t_i}^j > u_{t_{i'}}^j; i \neq i'$$

The preferences of the FNs are computed as per the Preference Generation of FN procedure (PGF), i.e., Algorithm 2).

The PGF takes as input a set of criteria $\mathcal{C}$, a set of tasks $t_i \in \mathcal{T}$, and the set of FNs $f_j \in \mathcal{F}$. It outputs the preference list of each FN f_j , i.e., $pref(f_j) \forall f_j \in \mathcal{F}$. PGF executes as follows. Initially, the preference list of each FN f_j is set to Φ . The FN f_j that has $|\mathcal{T}'| > 1$ invokes two algorithms: (i) AHP-based Criteria Weight (ACW) (Algorithm 3) and (ii) the TOPSIS-based Rank Determination (TRD) (Algorithm 7). The former obtains criteria

Algorithm 3: AHP-Based Criteria Weight (ACW).

Input: $\mathcal{C}$
Result: $\mathcal{W}$ ▷ Criteria weight vector
1 **Initialize:** $flag = false$
2 **if** $|\mathcal{C}| \geq 2$ **then**
3 $\mathbb{P} = \text{PMC}(\mathcal{C})$ ▷ $\mathbb{P}$ = Pairwise matrix
4 $\mathcal{W} = \text{CWD}(\mathbb{P}, \mathcal{C})$
5 **if** $|\mathcal{C}| \neq 2$ **then**
6 $flag = \text{CC}(\mathbb{P}, \mathcal{C}, \mathcal{W})$
7 **if** $flag == true$ **then**
8 return $\mathcal{W}$
9 **else**
10 **while** $flag \neq true$ **do** ▷ $\mathbb{P}$ is Updated
11 $\mathbb{P} = \text{PMC}(\mathcal{C})$
12 $\mathcal{W} = \text{CWD}(\mathbb{P}, \mathcal{C})$
13 $flag = \text{CC}(\mathbb{P}, \mathcal{C}, \mathcal{W})$
14 return $\mathcal{W}$
15 **else**
16 return $\mathcal{W}$
17 **else**
18 Return not applicable, exit

weights, whereas the latter takes criteria weights as input to rank all the tasks in $\mathcal{T}'$.

4) *AHP-Based Criteria Weight Determination:* Algorithm 3 takes the criteria set $\mathcal{C}$ as input and outputs the weight w_x corresponding to each criterion c_x , retained in a row vector $\mathcal{W}$. Algorithm 3 commences by initializing $flag$ to false. It then initiates the Pairwise Comparison Matrix Computation (PMC) procedure as per Algorithm 4. It assigns the relative weights of criteria taken as user inputs (Steps 2-10 of Algorithm 4). Note that the relative weights are assigned following the linear judgment scale [33]. Finally, PMC outputs a pairwise comparison matrix $\mathbb{P}$. After generating the pairwise comparison matrix, the Criteria Weight Determination (CWD) procedure (Algorithm 5) is invoked. Algorithm 5 first performs the column-wise normalization of $\mathbb{P}$ (Steps 2-4 of Algorithm 5). The normalized matrix $\bar{\mathbb{P}}$ is subsequently averaged row-wise to obtain a column vector $\mathcal{W}$ containing the criteria weights (Steps 5-7 of Algorithm 5). Once Algorithm 5 terminates, the Consistency Check (CC) procedure via Algorithm 6 is performed. The weights obtained via Algorithm 3 are auto consistent *iff* $|\mathcal{C}| = 2$, and consistency checking is not called for [33]. A consistency check is mandatory for all other cases and proceeds as follows. It encompasses the verification of the obtained weights within the allowable deviation range measured via consistency ratio $\mathbb{CR}$. Note that the random index with $\mathbb{RI}$ is set following the findings of [33]. If the consistency check passes, the $flag$ is set to true, and weights are retained in $\mathcal{W}$ (Steps 7-8 of Algorithm 3). Alternatively, inconsistent weight iteratively triggers Algorithms 4, 5, and 6 till a weight vector passes the consistency check (Steps 9-14 of Algorithm 3).

5) *TOPSIS-Based Task Ranking:* After obtaining the criteria weights, TRD, as depicted in Algorithm 7, is invoked. It takes as input (i) decision matrix $\mathbb{D}$, (ii) weight vector $\mathcal{W}$ (obtained via AHP), (iii) set of criteria $\mathcal{C}$ and (iv) set of alternatives

Algorithm 4: Pairwise-Comparison Matrix Computation (PMC).

Input: $\mathcal{C}$
Result: $\mathbb{P}$

```

1 Initialize :  $\mathbb{P} \in \mathbb{R}^{|\mathcal{C}| \times |\mathcal{C}|} \leftarrow O_{|\mathcal{C}| \times |\mathcal{C}|}$ 
2 for each  $c_x \in \mathcal{C}$  do
3   for each  $c_y \in \mathcal{C}$  do
4     if  $x == y$  then
5        $\mathbb{P}[c_x][c_y] = 1$   $\triangleright x = \text{Row Index}$ 
6     else if  $x < y$  then
7        $v_{c_x, c_y} = \text{GetRelativeImp}(c_x, c_y)$ 
8        $\mathbb{P}[c_x][c_y] = v_{c_x, c_y}$   $\triangleright y = \text{Column Index}$ 
9     else
10       $\mathbb{P}[c_x][c_y] = \frac{1}{\mathbb{P}[c_y][c_x]}$ 
11 return  $\mathbb{P}$ 

```

Algorithm 5: Criteria Weight Determination (CWD).

Input: $\mathbb{P}, \mathcal{C}$
Result: $\mathcal{W}$

```

1 Initialize :  $\bar{\mathbb{P}} \in \mathbb{R}^{|\mathcal{C}| \times |\mathcal{C}|} \leftarrow O_{|\mathcal{C}| \times |\mathcal{C}|}$ 
2 for each  $c_y \in \mathcal{C}$  do
3   for  $x = 1$  to  $|\mathcal{C}|$  do
4      $\bar{\mathbb{P}}[c_x][c_y] = \frac{\mathbb{P}[c_x][c_y]}{\sum_{x=1}^{|\mathcal{C}|} \mathbb{P}[c_x][c_y]}$ 
5 for each  $c_x \in \mathcal{C}$  do
6    $w_c = \frac{1}{|\mathcal{C}|} \sum_{y=1}^{|\mathcal{C}|} \bar{\mathbb{P}}[c_x][c_y]$ 
7    $\mathcal{W} = \mathcal{W} \cup w_c$ 
8 return  $\mathcal{W}$ 

```

Algorithm 6: Consistency Check (CC).

Input: $\mathbb{P}, \mathcal{C}, \mathcal{W}$
Result: $flag$: A boolean variable

```

1 Initialize :  $\mathbb{P}' \leftarrow O_{|\mathcal{C}| \times |\mathcal{C}|}$ ,  $flag = false$ 
2 for each  $c_y \in \mathcal{C}$  do
3   for  $x = 1$  to  $|\mathcal{C}|$  do
4      $\mathbb{P}'[c_x][c_y] = \mathbb{P}[c_x][c_y] * w_{c_y}$ 
5 for  $x = 1$  to  $|\mathcal{C}|$  do
6    $w'_{c_x} = \frac{\sum_{y=1}^{|\mathcal{C}|} \mathbb{P}'[c_y][c_x]}{w_{c_x}}$ 
7    $\mathbb{C}\mathbb{I} = \frac{\sum_{x=1}^{|\mathcal{C}|} w'_{c_x} - |\mathcal{C}|}{|\mathcal{C}| - 1}$   $\triangleright \mathbb{C}\mathbb{I} = \text{Consistency Index}$ 
8    $\mathbb{C}\mathbb{R} = \frac{\mathbb{C}\mathbb{I}}{\mathbb{R}\mathbb{I}}$   $\triangleright \mathbb{R}\mathbb{I} = \text{Random Index}$ 
9   if  $\mathbb{C}\mathbb{R} \leq 0.1$  then
10     $flag = true$ 
11 return  $flag$ 

```

$\mathcal{A}$ such that $\mathcal{A} \subseteq \mathcal{T}$. Algorithm 7 outputs a ranking for the alternatives and proceeds as follows. The decision matrix $\mathbb{D}$ is first column-wise normalized (Steps 2-5). Next, normalized matrix $\bar{\mathbb{D}}$ is then transformed into a weighted normalization matrix $\mathbb{D}'$ (Steps 6-8). The ideal best ($V_{c_y}^+$ or $V_{c_y}^-$) and worst ($V_{c_y}^-$

Algorithm 7: TOPSIS-Based Rank Determination (TRD).

Input: $\mathbb{D}, \mathcal{W}, \mathcal{C}, \mathcal{A}$ $\triangleright \mathcal{A} \subseteq \mathcal{T}$
Result: $\mathcal{U}$

```

1 Initialize :  $\bar{\mathbb{D}} \in \mathbb{R}^{|\mathcal{A}| \times |\mathcal{C}|} \leftarrow O_{|\mathcal{A}| \times |\mathcal{C}|}$ 
2 for each  $c_y \in \mathcal{C}$  do
3    $\bar{a}[c_y] = \sum_{t_i \in \mathcal{A}} (\mathbb{D}[t_i][c_y])^2$ 
4   for every  $b$  in  $c_y^{th}$  column of  $\mathbb{D}$  do
5      $\bar{\mathbb{D}}[b][c_y] = \frac{\mathbb{D}[b][c_y]}{\sqrt{\bar{a}[c_y]}}$ 
6 for each  $c_y \in \mathcal{C}$  do
7   for every  $b$  in  $c_y^{th}$  column of  $\bar{\mathbb{D}}$  do
8      $\mathbb{D}'[b][c_y] = \bar{\mathbb{D}}[b][c_y] * w_{c_y}$ 
9 for each  $c_y \in \mathcal{C}$  do
10  if  $c_y \in \mathcal{C} \mid c_y$  is a beneficial criterion in  $\mathbb{D}'$  then
11     $V_{c_y}^+ = \text{MaxCol}(c_y)$   $\triangleright V_{c_y}^+ = \text{Ideal best value}$ 
12     $V_{c_y}^- = \text{MinCol}(c_y)$   $\triangleright V_{c_y}^- = \text{Ideal worst value}$ 
13  else
14     $V_{c_y}^- = \text{MaxCol}(c_y)$   $\triangleright V_{c_y}^- = \text{Ideal best value}$ 
15     $V_{c_y}^+ = \text{MinCol}(c_y)$   $\triangleright V_{c_y}^+ = \text{Ideal worst value}$ 
16 for each  $t_i \in \mathcal{A}$  do
17    $u_{t_i}^+ = \sum_{y=1}^{|\mathcal{C}|} (\mathbb{D}'[t_i][c_y] - V_{c_y}^+)^2$ 
18    $u_{t_i}^- = \sum_{y=1}^{|\mathcal{C}|} (\mathbb{D}'[t_i][c_y] - V_{c_y}^-)^2$ 
19    $u_{t_i} = \frac{\sqrt{u_{t_i}^-}}{\sqrt{u_{t_i}^+} + \sqrt{u_{t_i}^-}}$ 
20    $\mathcal{U} = \mathcal{U} \cup u_{t_i}$ 
21  $\mathcal{U} = \text{List of } t_i \text{ in non-increasing order of } u_{t_i}$ 
22 return  $\mathcal{U}$ 

```

or $V_{c_y}^+$) values specific to each criterion $c \in \mathcal{C}$ are computed next (Steps 9-15). Note that the criterion for which a higher value is acceptable is beneficial. Then, the euclidean distance from the ideal best and worst denoted by $\sqrt{u_{t_i}^+}$ and $\sqrt{u_{t_i}^-}$ respectively is determined for each alternative $t_i \in \mathcal{A}$. Finally, the utility score u_{t_i} of an alternative $t_i \in \mathcal{A}$ is calculated and added into a global utility score vector $\mathcal{U}$ (Steps 16-20). The alternatives are then sequenced in non-increasing order of their utilities.

C. Computation of Precedence List

The precedence list (PL) contains all the tasks of $\mathcal{T}$ in non-increasing order of their utility scores. To generate the utility score of a task, we consider its (i) Computational demand, i.e., η_i , (ii) deadline, i.e., Δ_i , and (iii) feasible execution options, i.e., $|pref(t_i)|$. The consideration of computational demand allows the assignment of larger tasks onto computationally efficient FNs, assisting in reducing offloading delay. The deadline and feasible options boost the possibility of assignments, reducing the unassigned tasks. Therefore, considering the above-discussed parameters helps M-DAFTO achieve its design goals.

Considering multiple attributes drives us to adopt a hybrid AHP-TOPSIS-based strategy that outputs a ranking of tasks. In

Algorithm 8: Precedence List Construction (PLC).

Input: $\mathcal{T}, \mathcal{C}$
Result: PL

```

1 Initialize: PL =  $\Phi$ 
2  $\mathcal{W} = \text{ACW}(\mathcal{C})$ 
3 for each  $t_i \in \mathcal{T}$  do
4   for each  $c_x \in \mathcal{C}$  do
5     if  $x == 1$  then
6        $\mathbb{D}[t_i][c_x] = \eta_i$ 
7     else if  $x == 2$  then
8        $\mathbb{D}[t_i][c_x] = \Delta_i$ 
9     else
10       $\mathbb{D}[t_i][c_x] = |\text{pref}(t_i)|$ 
11 PL = TRD( $\mathbb{D}, \mathcal{W}, \mathcal{C}, \mathcal{T}$ )
12 return PL

```

Algorithm 9: Multistage Deferred Acceptance (MSDA).

Input: $\mathcal{T}, \mathcal{F}, \text{PL}; \text{pref}(t_i), \forall t_i \in \mathcal{T}; \text{pref}(f_j), h(f_j), l(f_j), \forall f_j \in \mathcal{F}$
Result: $\mathbb{M}$ $\triangleright$ Final Matching

```

1 Initialize:  $\Upsilon(t_i) = \Phi, \forall t_i \in \mathcal{T};$   

    $\Upsilon(f_j) = \Phi, k = 1, l^k(f_j) = l(f_j), h^k(f_j) = h(f_j),$   

    $\forall f_j \in \mathcal{F};$   

    $\mathbb{R}^0 = \mathcal{T}, \text{status} = \text{true}, \mathbb{M} = \Phi;$ 
2 for each  $t_i \in \mathcal{T}$  do
3    $\text{free}[t_i] = \text{true}$ 
4 while  $\text{PL} \neq \Phi$  do
5    $r^k = \sum_{j=1}^n l^k(f_j)$ 
6    $\mathbb{R}^k = \{t_{m-r^k+1}, t_{m-r^k+2}, \dots, t_m\}$ , Here  $\mathbb{R}^k$  is the set  

   that contains  $r^k$  tasks with lower preference  

   according to PL
7   if  $\mathbb{R}^{k-1} \setminus \mathbb{R}^k \neq \Phi$  then
8      $\mathbb{M}^k = \text{DA}(\mathbb{R}^{k-1} \setminus \mathbb{R}^k, \mathcal{F}, \{h(f_j)\}_{\forall f_j \in \mathcal{F}},$   

        $\{\text{pref}(t_i)\}_{\forall t_i \in \mathbb{R}^{k-1} \setminus \mathbb{R}^k}, \{\text{pref}(f_j)\}_{\forall f_j \in \mathcal{F}},$   

        $\text{status})$ 
9      $\mathbb{M} = \mathbb{M} \cup \mathbb{M}^k$ 
10     $\text{PL} = \text{PL} \setminus \{\mathbb{R}^{k-1} \setminus \mathbb{R}^k\}$ 
11   else
12      $\text{status} = \text{false}$ 
13      $\mathbb{T}' = \text{Tasks sorted in accordance with PL in } \mathbb{R}^k$ 
14      $\mathbb{M}^k = \text{DA}(\mathbb{T}', \mathcal{F}, \{l(f_j)\}_{\forall f_j \in \mathcal{F}}, \{\text{pref}(t_i)\}_{\forall t_i \in \mathbb{L}},$   

        $\{\text{pref}(f_j)\}_{\forall f_j \in \mathcal{F}}, \text{status})$ 
15      $\mathbb{M} = \mathbb{M} \cup \mathbb{M}^k$ 
16      $\text{PL} = \text{PL} \setminus \mathbb{R}^k$ 
17   for each  $f_j \in \mathcal{F}$  do
18      $h^{k+1}(f_j) = h^k(f_j) - |\Upsilon^k(f_j)|$ 
19      $l^{k+1}(f_j) = \max\{0, l^k(f_j) - |\Upsilon^k(f_j)|\}$ 
20    $k = k + 1$ 
21 return  $\mathbb{M}$ 

```

fact the ranking of tasks t_i and $t_{i'}$ follows (18).

$$t_i \succ_{\text{PL}} t_{i'} \iff u_{t_i} > u_{t_{i'}}; i \neq i' \quad (18)$$

The detailed working procedure of precedence list computation is explained in Algorithm 8.

D. Partially Stable Matching via MSDA Algorithm

The overall working of the MSDA algorithm is shown in Algorithm 9. The inputs to the algorithm are: (i) sets $\mathcal{T}$ and $\mathcal{F}$, (ii) a precedence list, PL, (iii) maximum and minimum quota, $h(f_j)$ and $l(f_j)$, $\forall f_j \in \mathcal{F}$, and (iv) preferences of agents in $\mathcal{T} \cup \mathcal{F}$. The algorithm outputs a *partially stable matching* $\mathbb{M}$ that comprises matched pairs (t_i, f_j) . The algorithm starts by initializing the matching of agents in $\mathcal{T} \cup \mathcal{F}$ to Φ . As the matching procedure iterates over multiple stages, we define an iterator k that tracks the executing stage of the MSDA algorithm. Prior to stage 1, $\mathbb{R}^0$ is initialized to $\mathcal{T}$ and $h^k(f_j)$ and $l^k(f_j)$ are respectively initialized to maximum quota $h(f_j)$ and minimum quota $l(f_j)$ of f_j . Each task t_i is set to be free, indicating its availability in the matching procedure (Steps 2-3 of Algorithm 9). MSDA procedure then extracts the least preferred r^k tasks in PL and retains them in $\mathbb{R}^k$ (Steps 5-6 of Algorithm 9). Depending on $\mathbb{R}^{k-1} \setminus \mathbb{R}^k$, MSDA procedure triggers Algorithm 10 (DA) either with maximum or minimum quota. The existence of at least one task in $(\mathbb{R}^{k-1} \setminus \mathbb{R}^k)$ indicates the availability of additional tasks $(\mathbb{R}^{k-1} \setminus \mathbb{R}^k)$ after reservation. Hence, DA is triggered for these additional tasks with maximum quotas of FNs (Steps 7-10 of Algorithm 9). On the other hand, if $\mathbb{R}^{k-1} \setminus \mathbb{R}^k$ equals Φ , DA is invoked for $\mathbb{R}^k$ tasks with minimum quotas of FNs (Steps 11-16 of Algorithm 9). After executing DA, the maximum and minimum quota of FNs are updated, and upon exhaustion of PL, the final matching $\mathbb{M}$ is returned. Algorithm 10 enters the proposal phase, wherein an unassigned task t_i proposes to its most preferred FN $f_{j'}$. Depending on the updated quota $s^k(f_{j'})$ of $f_{j'}$ either of the following cases may be encountered: (i) it has sufficient resources (ii) it does not have enough resources. In the former, t_i and $f_{j'}$ are matched (Steps 6-10 of Algorithm 10). The latter is subdivided into two cases depending on the preference of t_i at $f_{j'}$. If $t_{i'} \succ_{f_{j'}} t_i$, where $t_{i'}$ is the worst assigned task, t_i is rejected. Alternatively, $t_i \succ_{f_{j'}} t_{i'}$, t_i replaces $t_{i'}$ and in the present iteration, the addition of the task $t_{i'}$ to the set $\mathbb{T}$ provides an opportunity for assignment (Steps 12-23 of Algorithm 10). Note that a true value for the status variable DA allows for replacement; otherwise, it does not. The motivation for disallowing replacements is to abide by the minimum quota constraints imposed [30]. During the last iteration, *M-DAFTO* focuses on fulfilling the minimum quotas at FNs by allocating all tasks designated for the minimum quota stage from the PL. Nevertheless, this compulsory allocation could result in an unfair distribution, potentially introducing blocking pairs, as defined in Definition 4. Such situations may prompt individuals to deviate from their existing agent allocation. To mitigate this concern and promote fairness in the distribution of tasks and FNs according to the PL, the concept of *PL-fairness* is introduced. It aims to eliminate incentives for participants to alter their allocations.

1) *A Toy Example:* To illustrate the working of *M-DAFTO*, we consider an example with five tasks $\{t_1, t_2, t_3, t_4, t_5\}$, and three FNs $\{f_1, f_2, f_3\}$. The preference profiles of tasks, $\text{pref}(t_i)$, $\forall i \in [1, 5]$ and that of FNs, $\text{pref}(f_j)$, $\forall j \in [1, 3]$ are shown in Tables II and III respectively. Table III also captures the maximum and minimum quotas of each f_j , denoted by $h(f_j)$ and $l(f_j)$, respectively. Additionally, we

Algorithm 10: Deferred Acceptance (DA).

Input: $\mathbb{T}, \mathcal{F}; s^k(f_j), \forall f_j \in \mathcal{F}; pref(t_i), \forall t_i \in \mathbb{T}; pref(f_j), \forall f_j \in \mathcal{F}; status$

Result: $\mathbb{M}^k$

```

1 Initialize:  $\mathbb{M}^k = \Phi$ 
2 while  $\exists t_i \mid free[t_i] \text{ and } pref(t_i) \neq \Phi$  do
3    $f_{j'} = \text{highest ranked FN in } pref(t_i) \text{ to which } t_i \text{ has}$ 
   not proposed yet.
4   Send proposal to  $f_{j'}$ 
5   Remove  $f_{j'}$  from  $pref(t_i)$ 
6   if  $s^k(f_{j'}) > 0$  then
7      $\Upsilon(f_{j'}) = \Upsilon(f_{j'}) \cup \{t_i\}$ 
8      $\mathbb{M}^k = \mathbb{M}^k \cup \{(t_i, f_{j'})\}$ 
9      $free[t_i] = \text{false}$ 
10     $s^k(f_{j'}) = s^k(f_{j'}) - 1$ 
11   else
12     if  $status \neq \text{false}$  then
13        $t_{i'} = \text{Worst task currently matched in } \Upsilon(f_{j'})$ 
14       if  $t_i \succ_{f_{j'}} t_{i'}$  then
15          $\Upsilon(f_{j'}) = \Upsilon(f_{j'}) \setminus \{t_{i'}\}$ 
16          $free[t_{i'}] = \text{true}$ 
17          $\mathbb{T} = \mathbb{T} \cup \{t_{i'}\}$ 
18          $\mathbb{M}^k = \mathbb{M}^k \setminus \{(t_{i'}, f_{j'})\}$ 
19          $\Upsilon(f_{j'}) = \Upsilon(f_{j'}) \cup \{t_i\}$ 
20          $free[t_i] = \text{false}$ 
21          $\mathbb{M}^k = \mathbb{M}^k \cup \{(t_i, f_{j'})\}$ 
22       else
23         Reject  $t_i$ .
24     else
25       Reject  $t_i$ .
26 return  $\mathbb{M}^k$ 

```

TABLE II
PREFERENCES OF TASKS

t_i	$pref(t_i)$
t_1	$f_2 \ f_1 \ f_3$
t_2	$f_2 \ f_1 \ f_3$
t_3	$f_1 \ f_2 \ f_3$
t_4	$f_2 \ f_3 \ f_1$
t_5	$f_1 \ f_2 \ f_3$

consider a PL that follows the ordering $\{t_1, t_2, t_3, t_4, t_5\}$. Initially, $\mathbb{R}^0 = \{t_1, t_2, t_3, t_4, t_5\}$. Algorithm 9 executes in multiple stages tracked by the variable k . At stage k of MSDA, Algorithm 10 is invoked with either maximum or minimum quota of the FNs depending upon the number of unprocessed tasks captured by $|\mathbb{R}^{k-1} \setminus \mathbb{R}^k|$. If there are no unprocessed tasks, i.e., $|\mathbb{R}^{k-1} \setminus \mathbb{R}^k| = 0$, DA is invoked with minimum quotas, otherwise with maximum quotas. In the first stage ($k = 1$), $r^1 = 3$, hence r^1 least preferred tasks, i.e., $\{t_3, t_4, t_5\}$ are reserved from PL and retained in $\mathbb{R}^1$ for processing at a later stage. As $\mathbb{R}^0 \setminus \mathbb{R}^1$ is not empty, Algorithm 10 is invoked for tasks $\{t_1, t_2\}$ with maximum quotas of FNs. Following Algorithm 10, tasks

TABLE III
PREFERENCES OF FNS

f_j	$pref(f_j)$	$h(f_j)$	$l(f_j)$
f_1	$t_5 \ t_3 \ t_1 \ t_2 \ t_4$	2	1
f_2	$t_3 \ t_4 \ t_1 \ t_2 \ t_5$	3	1
f_3	$t_3 \ t_4 \ t_2 \ t_5 \ t_1$	1	1

t_1 and t_2 send proposals to their most preferred FN f_2 . The proposals are accommodated at f_2 , and available resources at f_2 denoted by $s^1(f_2)$ are updated to 1. Note that the processed tasks are removed from PL. Hence, the updated PL at the end stage 1 contains $\{t_3, t_4, t_5\}$. Additionally, Stage 1 terminates by updating the current maximum quota $h^2(f_2)$ to 1 and minimum quota $l^2(f_2)$ to 0. MSDA then enters the second stage ($k = 2$), where $r^2 = 2$ and $\mathbb{R}^2 = \{t_4, t_5\}$. As $\mathbb{R}^1 \setminus \mathbb{R}^2 \neq \Phi$, DA is invoked for task t_3 with maximum quotas at FNs. t_3 proposes to its most preferred FN f_1 and is matched. Hence, the available resources at f_1 denoted by $s^2(f_1)$ is updated to 1. Then MSDA updates PL, maximum quota $h^3(f_1)$, and minimum quota $l^3(f_1)$ to $\{t_4, t_5\}$, 1, and 0 respectively. MSDA executes its third stage ($k = 3$), where $r^3 = 1$ and $\mathbb{R}^3 = \{t_5\}$. DA is triggered for the only task t_4 which is subsequently matched to f_2 and $s^3(f_2)$ and PL are updated to 0 and $\{t_5\}$, respectively. Here, both maximum quota $h^3(f_3)$ and minimum quota $l^3(f_3)$ are updated to 0. In the fourth stage, $r^4 = 1$, $\mathbb{R}^4 = \{t_5\}$, $\mathbb{R}^4 \setminus \mathbb{R}^3 = \Phi$, hence DA is invoked for t_5 with minimum quotas. f_1, f_2 reject t_5 owing to exhaustion of minimum quotas. Therefore, t_5 is finally matched to f_3 , which terminates the MSDA. The final matching $\mathbb{M} = \{(t_1, f_2), (t_2, f_2), (t_3, f_1), (t_4, f_2), (t_5, f_3)\}$.

Theorem 1. (PL-Fairness): MSDA produces a PL-fair matching between the set of tasks $\mathcal{T}$ and FNs $\mathcal{F}$.

Proof: The proof can be found in Appendix D.1, available online. $\square$

Theorem 2. (Individual Rationality): MSDA produces a individual rationality matching between the set $\mathcal{T}$ and $\mathcal{F}$.

Proof: The proof is detailed in Appendix D.2, available online. $\square$

Theorem 3. (Partially Stable): MSDA produces a partially stable matching between the set of tasks $\mathcal{T}$ and FNs $\mathcal{F}$.

Proof: Since MSDA satisfies the properties of PL-fairness and individual rationality uniquely, the matching generated by it is partially stable. $\square$

Theorem 4. (Time Complexity): M-DAFTO converges with the time complexity of $O(m * n)$, where m and n are the total numbers of tasks and FNs, respectively.

Proof: The proof can be found in Appendix D.3, available online. $\square$

Theorem 5. (NP-Hardness): The offloading problem in M-DAFTO is NP-Hard.

Proof: Appendix D.4, available online, contains the proof for NP-Hardness. $\square$

VI. PERFORMANCE EVALUATION

For simulation, the iFogSim simulator has been used [34]. In this section, we have discussed the environmental setup and elaborately analyzed the simulation results.

A. Environmental Setup

The IoT devices and FNs are deployed statically, and the distance between an IoT and a FN is uniformly distributed $U[50, 500]$ meters. The uplink and downlink bandwidth is 20 MHz [15]. For experimentation, we consider 4 test scenarios, S-1, S-2, S-3, and S-4 for 250, 500, 750, and 1000 tasks and 5 FNs. Further, we perform 10 test runs for each scenario, and we examine the comparative behavior of protocols. The input size, computational requirement, deadline, and output size are generated uniformly in the range $U[300, 600]$ Kb, $U[210, 480]$ million cycles, and $U[15, 22]$ s, and $U[10, 20]$ Kb, respectively [1]. Moreover, the transmission power of d_i and noise power are set to 0.5 W and 10^{-10} W respectively [15]. FN parameters, such as computational resources and number of VRUs, are uniformly set in $U[6, 10]$ GHz and $U[200, 500]$, respectively, with each FN having a transmission power of 1 W. The free space path loss between an IoT and a FN f_j in PCS-1900 GSM band is determined as $pl_{d_i, f_j} = 38.02 + 20\log_{10}(dist(d_i, f_j))$ dB, where $dist(d_i, f_j)$ represents the distance between an IoT and a FN. Considering pl_{d_i, f_j} , the channel gain $h_{i,j}$ is derived as $h_{i,j} = 10^{-(pl_{d_i, f_j})/10}$.

B. The Baseline Algorithms

We have considered the following baselines to compare the performance of *M-DAFTO*.

- A matching theory framework for tasks offloading in fog computing for IoT systems (MBTO) [13]: It aims to minimize the total offloading delay by reducing the worst completion times of tasks. Offloading is modeled as a matching between IoTs and FNs, with a maximum quota at the FNs. To obtain a stable assignment, the authors adopt a DA algorithm variant.
- LETO [15]: This work reduces the total offloading delay and number of outages in an IoT-Fog setup. The offloading is formulated as a matching game with maximum and minimum quotas at the FNs. A multistage deferred acceptance (MSDA) algorithm generates a stable offloading schedule.
- Heuristic-based task offloading (HTO) [12]: It randomly offloads tasks in a non-increasing order of their input size to FNs.
- GPM [14]: It greedily assigns the tasks to the FNs having the maximum number of VRUs.

C. Simulation Results

M-DAFTO's performance is validated considering three metrics, i.e., (i.) Performance-based; (ii.) Matching-based; and (iii.) Utilization-based. Subsequently, we provide a detailed insight into each category.

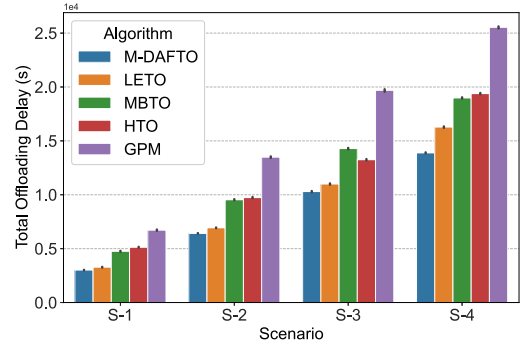


Fig. 3. Total offloading delay (s) versus scenarios.

1) *Performance-Based Metrics*: The primary evaluation metrics, such as *total offloading delay* and *outages*, are studied in this category.

Total Offloading Delay: Fig. 3 captures the aggregate offloading delay in executing offloaded tasks at the FNs across different scenarios. The offloading delay escalates with increasing tasks, which is anticipated. However, *M-DAFTO* outperforms the baselines across scenarios by achieving minimum offloading delay. This is attributed to the following reasons. *First*, the residual time of larger tasks¹ is often smaller owing to its higher computational demand, resulting in limited offloading options, hence higher urgency (17). *Second*, the PL dictates the processing order of the tasks and favors the offloading of larger tasks having a smaller deadline (18). Although LETO is similar to *M-DAFTO*, the former differs in PL-ordering and FN preferences based on the task's deadline. This results in the preferable assignment of tasks with a smaller deadline to computationally efficient FNs.² Moreover, a smaller deadline does not necessarily imply a smaller task. Hence, processing based on deadlines can push larger tasks onto computationally inferior FNs, elevating the offloading delay. MBTO, HTO, and GPM exhibit inferior performance compared to *M-DAFTO* and LETO, as the IoTs do not consider the FNs' computational capabilities while generating the offloading schedule. Considering the comparative behavior of MBTO and HTO, the former outperforms the latter. However, we observe a flipped behavior for scenario S-3 due to the random processing order of the FNs. Conversely, GPM has the worst offloading delay due to greedy task offloading to FNs with the highest VRUs.

Outages: It is the difference between tasks ($\mathcal{T}$) in the system and successfully offloaded ones, which can be calculated per (19).

$$\mathcal{O} = \{\mathcal{T}\} \setminus \{\cup_{j=1}^n \Upsilon(f_j)\} \quad (19)$$

Fig. 4 illustrates the number of outages experienced by different strategies across test scenarios. It can be inferred from Fig. 4 that the number of outages increases with increasing tasks. *M-DAFTO* outperforms the baselines owing to the following reasons. *First*, tasks rank the FNs in non-decreasing order of the

¹Larger task imply a higher CPU demand

²Computationally efficient FNs have higher compute capabilities for the VRUs.

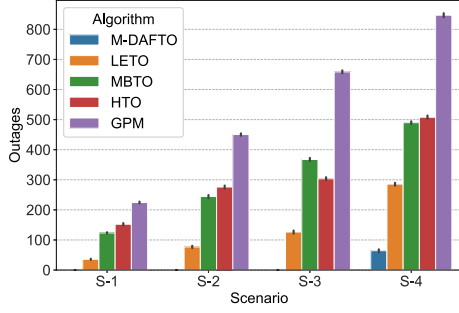


Fig. 4. Outages versus scenarios.

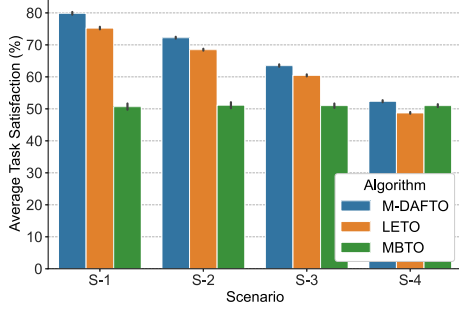


Fig. 5. Average task satisfaction (%) versus scenarios.

offloading delays. *Second*, the tasks are processed at the FNs based on their urgencies (refer to (17)). *Third*, the processing order of the tasks enforced by the PL assists in pushing the larger tasks onto computationally efficient FNs. The combined impact of the policies as mentioned above assists *M-DAFTO* in achieving minimal outages. *LETO* exhibits inferior performance compared to *M-DAFTO* as tasks having limited FN options for successful execution are not prioritized during the assignment. Large tasks may not have a smaller deadline; hence, their processing is deferred in *LETO*. This results in their assignments to inferior FNs due to limited capacities at faster FNs. *MBTO*, *HTO*, and *GPM* are inconsiderate towards the FNs' computational capabilities while generating IoT preferences and deadlines for the tasks while generating the FN preferences.

2) Matching-Based Metrics: A primary metric to evaluate the performance of matching-based techniques is the *satisfaction factor* of individual agents. It reflects an agent's happiness with its match, considering its preference list. Ideally, agents matched to their more preferred choices have higher satisfaction.

Task Satisfaction Factor: Fig. 5 indicates the average satisfaction factor of tasks for different matching-based strategies used for comparison. Given that t_i is matched to f_j in Υ , satisfaction factor $S_{i,j}(\%)$ of t_i can be computed as per (20). Here, $rank(f_j)$ captures the position of f_j in $pref(t_i)$. The detailed computation of the satisfaction of tasks for an example scenario is provided in Appendix C.1, available online.

$$S_{i,j}(\%) = \frac{|pref(t_i)| - rank(f_j)}{|pref(t_i)|} * 100 \quad (20)$$

It can be observed from Fig. 5 that *M-DAFTO* achieves the highest task satisfaction compared to *MBTO* and *LETO*. This is attributed to the fact that *M-DAFTO* executes the majority of the

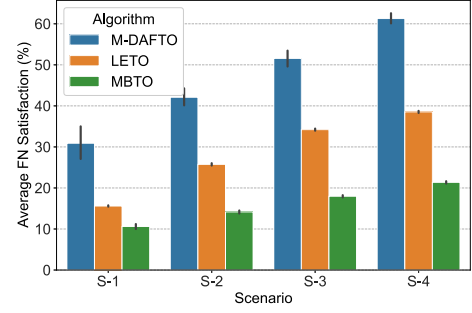


Fig. 6. Average FN satisfaction (%) versus scenarios.

offloaded tasks within their prescribed deadlines (refer to Fig. 4), thereby achieving minimal outages. A task experiencing an outage has zero satisfaction; hence, excessive outages deleteriously impact the average task satisfaction. As it can be inferred from Fig. 4, *LETO* suffers more outages than *M-DAFTO*, resulting in lower task satisfaction. On the other hand, *MBTO* achieves the lowest satisfaction due to the maximum outages suffered during offloading. However, task satisfaction in *MBTO* is relatively consistent across scenarios. This is because *MBTO* offloads the tasks to the FNs in close proximity, and a uniformly distributed setup assists the protocol in achieving a relatively distributed assignment.

FN Satisfaction Factor: Fig. 6 illustrates the average satisfaction factor of FNs for different matching-based strategies. Given a matching Υ , let $\Upsilon(f_j)$ be the set of tasks matched to f_j . The satisfaction factor $S_{j,i}(\%)$ captures the satisfaction obtained by f_j in executing the offloaded task t_i and is computed as per (21) and (22). Here, *count* tracks the number of currently matched agents per $pref(f_j)$. A detailed insight into the computation of FN satisfaction is provided in Appendix C.2, available online.

$$S_{j,i}(\%) = \frac{|pref(f_j)| - rank(t_i)}{|pref(f_j)| - count} * 100; \forall t_i \in \Upsilon(f_j) \quad (21)$$

$$S_j(\%) = \frac{\sum_{\forall i \in [1,m] \& t_i \in \Upsilon(f_j)} S_{j,i}}{\min(h(f_j), |pref(f_j)|)}; \forall j \in [1,n] \quad (22)$$

It can be observed from Fig. 6 that the satisfaction of FNs escalates with increasing tasks as the $|pref(f_j)|$ increases, consequently boosting the satisfaction factor of similar options. It can be inferred from Fig. 6 that *M-DAFTO* achieves the highest satisfaction as it experiences minimal outages. Moreover, the number of outages dictates the satisfaction of FNs for different protocols, i.e., the higher the number of outages, the lower its satisfaction.

3) Utilization-Based Metric: We also provide an insight into the load distribution (used VRUs) of different schemes and study their comparative behavior.

Resource Utilization: Fig. 7(a)–(d) highlight the utilization of FNs for different schemes across test cases. The utilization of FNs increases for increasing tasks, which is anticipated. The FNs are utilized in non-increasing order of their computational capabilities of VRUs (refer to Fig. 1 of Appendix E, available online) owing to the setting of the preferences for IoTs based on their offloading delays (refer to (16)). However, due to the

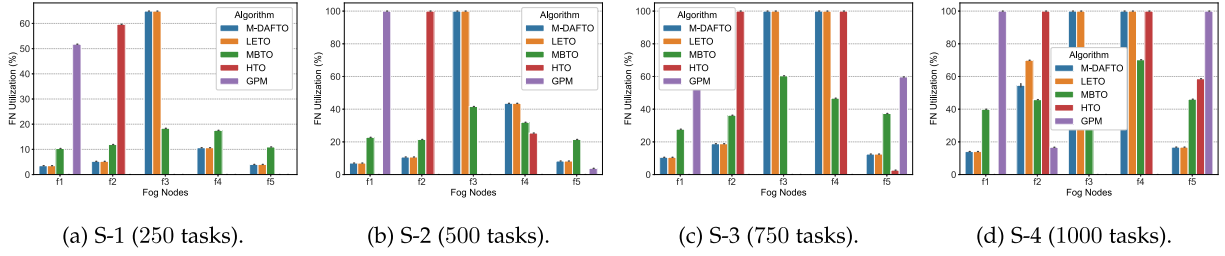


Fig. 7. FN resources utilization across scenarios.

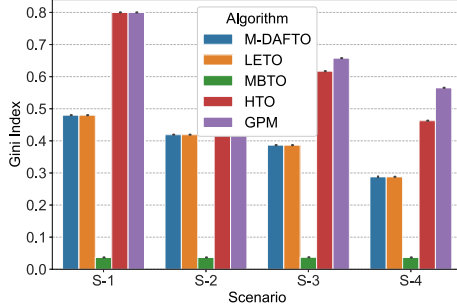


Fig. 8. Gini index versus scenarios.

imposition of minimum quota in *M-DAFTO* and *LETO*, all the FNs are assigned some tasks. *MBTO* obtains a relatively distributed assignment compared to the baselines as the tasks are preferably offloaded to the FNs in its proximity in a uniformly distributed IoT-Fog network. In *HTO*, tasks are assigned FNs in a predefined random sequence, enforcing a clustered assignment of FNs appearing at the beginning of the random sequence. Alternatively, *GPM* greedily offloads tasks to the FNs in non-increasing order of their VRUs, resulting in a clustered assignment at larger FNs.

Gini Index: It indicates the statistical dispersion or inequality measure of load across FNs [35]. The Gini index can be computed as per (23), where $\bar{x} = \sum_{j=1}^n |\Upsilon(f_j)|/n$. It lies in the 0 – 1; a 0 value represents perfect equality, whereas 1 indicates perfect inequality.

$$Gini(n) = \frac{1}{2n^2\bar{x}} \sum_{j=1}^n \sum_{k=1}^n ||\Upsilon(f_j)| - |\Upsilon(f_k)|| \quad (23)$$

Fig. 8 illustrates the Gini index across strategies for different test scenarios. *MBTO* is the best protocol as it achieves a relatively dispersed allocation owing to the nature of offloading in *MBTO* and the uniformly distributed deployment of IoT devices. Moreover, *M-DAFTO* and *LETO* achieve the same load dispersion due to similar allocation policies. The Gini index of *M-DAFTO* and *LETO* is better than *HTO* and *GPM*. This is because of the imposition of a minimum quota at FN in *M-DAFTO* and *LETO*, which achieves non-clustered allocations. Moreover, the Gini index of *HTO* is better than *GPM* as it performs greedy assignments by selecting random FNs, resulting in clustered allocation. On the other hand, *GPM*'s Gini index is the worst as it performs greedy task assignments to FNs in decreasing

order of their VRUs (maximum quota), resulting in clustered allocation.

Appendix E, available online, includes some additional simulation results regarding the Jain fairness index, worst offloading delay, and worst number of outages.

VII. CONCLUSIONS & FUTURE DIRECTIONS

This paper proposed a framework *M-DAFTO* that generates an efficient offloading policy for the IoT-Fog interconnection network. The offloading is modeled as a matching game with maximum and minimum quotas at the FNs. To overcome the shortcomings of the DA and operate with minimum quotas, we adopted the MSDA to solve the problem mentioned above. Further, the MSDA generates a *partial stable* assignment between IoTs and FNs in a polynomial time. Simulation results confirm that the proposed scheme achieves 30.26 % and 93.53 % reduction of total offloading delay and outages, respectively, compared to the baselines. Though *M-DAFTO* achieves enhanced performance, the agents' preferences are immutable. The work can be extended to consider dynamic preferences, including additional parameters such as offloading energy and channel interference. Moreover, the offloading setup can also be supplemented by including cloud resources in the IoT-Fog interconnection network.

REFERENCES

- [1] C. Swain et al., "METO: Matching-theory-based efficient task offloading in IoT-Fog interconnection networks," *IEEE Internet Things J.*, vol. 8, no. 16, pp. 12 705–12 715, Aug. 2021.
- [2] I. Sarkar, M. Adhikari, N. Kumar, and S. Kumar, "A collaborative computational offloading strategy for latency-sensitive applications in fog networks," *IEEE Internet Things J.*, vol. 9, no. 6, pp. 4565–4572, Mar. 2022.
- [3] M. Adhikari, M. Mukherjee, and S. N. Srirama, "DPTO: A deadline and priority-aware task offloading in fog computing framework leveraging multilevel feedback queueing," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 5773–5782, Jul. 2020.
- [4] A. Kishor and C. Chakrabarty, "Task offloading in fog computing for using smart ant colony optimization," *Wireless Pers. Commun.*, vol. 127, pp. 1683–1704, 2022.
- [5] C. Swain, M. N. Sahoo, A. Satpathy, K. Muhammad, S. Bakshi, and J. P. C. Rodrigues, "A-DAFTO: Artificial cap deferred acceptance based fair task offloading in complex IoT-fog networks," *IEEE Trans. Consum. Electron.*, vol. 69, no. 4, pp. 914–926, Nov. 2023.
- [6] L. Jiao, R. Friedman, X. Fu, S. Secci, Z. Smoreda, and H. Tschofenig, "Cloud-based computation offloading for mobile devices: State of the art, challenges and opportunities," in *Proc. Future Netw. Mobile Summit*, 2013, pp. 1–11.
- [7] C. Swain, M. N. Sahoo, and A. Satpathy, "SPATO: A student project allocation based task offloading in IoT-fog systems," in *Proc. IEEE Int. Conf. Commun.*, 2021, pp. 1–6.

- [8] C. Canali and R. Lancellotti, "GASP: Genetic algorithms for service placement in fog computing systems," *Algorithms*, vol. 12, no. 10, 2019, Art. no. 201.
- [9] Q. Wang and S. Chen, "Latency-minimum offloading decision and resource allocation for fog-enabled Internet of Things networks," *Trans. Emerg. Telecommun. Technol.*, vol. 31, no. 12, 2020, Art. no. e3880.
- [10] S. A. Zakaryia, S. A. Ahmed, and M. K. Hussein, "Evolutionary offloading in an edge environment," *Egyptian Inform. J.*, vol. 22, no. 3, pp. 257–267, 2021.
- [11] Y. Deng, P. Wang, and L. Li, "Task offloading for mixed cloud/fog computing in vehicular network using genetic particle swarm optimization," in *Proc. 4th Int. Conf. Inf. Commun. Signal Process.*, 2021, pp. 489–494.
- [12] H. Bao and Y. Liu, "A two-sided matching approach for distributed edge computation offloading," in *Proc. IEEE/CIC Int. Conf. Commun. China*, 2019, pp. 535–540.
- [13] F. Chiti, R. Fantacci, and B. Picano, "A matching theory framework for tasks offloading in fog computing for IoT systems," *IEEE Internet Things J.*, vol. 5, no. 6, pp. 5089–5096, Dec. 2018.
- [14] F. Chiti, R. Fantacci, F. Paganelli, and B. Picano, "Virtual functions placement with time constraints in fog computing: A matching theory perspective," *IEEE Trans. Netw. Service Manag.*, vol. 16, no. 3, pp. 980–989, Sep. 2019.
- [15] C. Swain, M. N. Sahoo, and A. Satpathy, "LETO: An efficient load balanced strategy for task offloading in IoT-fog systems," in *Proc. IEEE Int. Conf. Web Serv.*, 2021, pp. 459–464.
- [16] M. Mukherjee, S. Kumar, M. Shojafar, Q. Zhang, and C. X. Mavromoustakis, "Joint task offloading and resource allocation for delay-sensitive fog networks," in *Proc. IEEE Int. Conf. Commun.*, 2019, pp. 1–7.
- [17] J. He, "Optimization of edge delay sensitive task scheduling based on genetic algorithm," in *Proc. Int. Conf. Algorithms Data Mining Inf. Technol.*, 2022, pp. 155–159.
- [18] Z. Li and Q. Zhu, "Genetic algorithm-based optimization of offloading and resource allocation in mobile-edge computing," *Information*, vol. 11, no. 2, 2020, Art. no. 83.
- [19] A. Abouaomar, A. Kobbane, and S. Cherkaoui, "Matching-game for user-fog assignment," in *Proc. IEEE Glob. Commun. Conf.*, 2018, pp. 1–6.
- [20] M. Ali, N. Riaz, M. I. Ashraf, S. Qaisar, and M. Naem, "Joint cloudlet selection and latency minimization in fog networks," *IEEE Trans. Ind. Informat.*, vol. 14, no. 9, pp. 4055–4063, Sep. 2018.
- [21] D. Nowak, T. Mahn, H. Al-Shatri, A. Schwartz, and A. Klein, "A generalized nash game for mobile edge computation offloading," in *Proc. 6th IEEE Int. Conf. Mobile Cloud Comput. Serv. Eng.*, 2018, pp. 95–102.
- [22] P. Kaur and S. Mehta, *Improvement of Task Offloading for Latency Sensitive Tasks in Fog Environment*. Singapore: Springer, 2022, pp. 49–63.
- [23] Q. Luo, C. Li, T. H. Luan, and W. Shi, "Minimizing the delay and cost of computation offloading for vehicular edge computing," *IEEE Trans. Services Comput.*, vol. 15, no. 5, pp. 2897–2909, Sep./Oct. 2022.
- [24] Y. Pan, H. Jiang, H. Zhu, and J. Wang, "Latency minimization for task offloading in hierarchical fog-computing C-RAN networks," in *Proc. IEEE Int. Conf. Commun.*, 2020, pp. 1–6.
- [25] M. Mukherjee, V. Kumar, S. Kumar, J. Lloret, Q. Zhang, and M. Guo, "Reconfigurable intelligent surface-assisted edge computing to minimize delay in task offloading," in *Proc. IEEE Glob. Commun. Conf.*, 2021, pp. 01–06.
- [26] P. Liu, J. Li, and Z. Sun, "Matching-based task offloading for vehicular edge computing," *IEEE Access*, vol. 7, pp. 27 628–27 640, 2019.
- [27] B. Yang, M. Sun, X. Hong, and X. Guo, "A deadline-aware offloading scheme for vehicular fog computing at signalized intersection," in *Proc. IEEE Int. Conf. Pervasive Comput. Commun. Workshops*, 2020, pp. 1–6.
- [28] Y. Sun, J. Song, S. Zhou, X. Guo, and Z. Niu, "Task replication for vehicular edge computing: A combinatorial multi-armed bandit based approach," in *Proc. IEEE Glob. Commun. Conf.*, 2018, pp. 1–7.
- [29] D. Gale and L. S. Shapley, "College admissions and the stability of marriage," *Amer. Math. Monthly*, vol. 69, no. 1, pp. 9–15, 1962.
- [30] D. Fragiadakis, A. Iwasaki, P. Troyan, S. Ueda, and M. Yokoo, "Strategyproof matching with minimum quotas," *ACM Trans. Econ. Computation*, vol. 4, no. 1, pp. 1–40, 2016.
- [31] A. Satpathy, M. N. Sahoo, L. Behera, and C. Swain, "ReMatch: An efficient virtual data center re-matching strategy based on matching theory," *IEEE Trans. Services Comput.*, vol. 16, no. 2, pp. 1373–1386, Mar./Apr. 2023.
- [32] O. Semiar, W. Saad, and M. Bennis, "Downlink cell association and load balancing for joint millimeter wave-microwave cellular networks," in *Proc. IEEE Glob. Commun. Conf.*, 2016, pp. 1–6.
- [33] A. Satpathy, M. N. Sahoo, A. K. Sangaiah, C. Swain, and S. Bakshi, "CoMap: An efficient virtual network re-mapping strategy based on coalitional matching theory," *Comput. Netw.*, vol. 216, 2022, Art. no. 109248.
- [34] H. Gupta, A. V. Dastjerdi, S. K. Ghosh, and R. Buyya, "iFogSim: A toolkit for modeling and simulation of resource management techniques in the Internet of Things, edge and fog computing environments," *Softw. Pract. Experience*, vol. 47, no. 9, pp. 1275–1296, 2017.
- [35] Z. Ning, L. Guo, Y. Peng, and X. Wang, "Joint scheduling and routing algorithm with load balancing in wireless mesh network," *Comput. Elect. Eng.*, vol. 38, no. 3, pp. 533–550, 2012.



Chittaranjan Swain (Graduate Student Member, IEEE) received the MTech degree in computer science engineering from VIT University, Vellore, India. He is a computer science and engineering research scholar with NIT Rourkela, India. His research interests include cloud computing, fog computing, and Internet of Things.



Manmath Narayan Sahoo (Member, IEEE) received the PhD degree in computer science from NIT Rourkela, India, in 2014. He is an assistant professor with CSE, NIT Rourkela, India. He has been a reviewer, guest editor, track chair, and program chair in many journals and conferences. His research interests include mobile Ad-hoc networks, sensor networks, and cloud computing.



Anurag Satpathy (Graduate Student Member, IEEE) received the PhD degree in computer science and engineering from the National Institute of Technology, Rourkela, India. He is a post-doctoral fellow with the Department of Computer Science, Missouri University of Science and Technology, Rolla, MO, USA. His research interests include cloud computing, the Internet of Things, and distributed systems.



Sambit Bakshi (Senior Member, IEEE) is currently with the Department of Computer Science and Engineering, National Institute of Technology Rourkela, India. His area of interests include surveillance, biometric security, and digital forensics. He presently serves as associate editor of the *IEEE Transactions on Computational Social Systems*, *IEEE Transactions on Technology and Society*, *IEEE Transactions on Education*, *IEEE Systems Journal*, *Innovations in Systems and Software Engineering* Springer: A NASA Journal, *Expert Systems with Applications*, *Engineering Applications of Artificial Intelligence*, *Image and Vision Computing*, *Multimedia Systems*, *Multimedia Tools and Applications*, and *Neural Processing Letters*. He previously served as associate editor of the *IEEE IT Professional Magazine*, *IEEE Access*, *PLoS One*, and *Expert Systems* in the past. He is founding chair of IEEE Rourkela Subsection and IEEE Kolkata Section Biometrics Council Chapter. He serves as a distinguished lecturer of IEEE Systems Council for 2023–2025. He has published widely in more than 100 journals and conferences.



Soumya K. Ghosh (Senior Member, IEEE) received the PhD degree in computer science and engineering from the IIT, Kharagpur. Currently, he is a professor with the Department of Computer Science and Engineering, IIT Kharagpur. Prior to joining IIT Kharagpur, he was with the ISRO, working in the area of satellite remote sensing and GIS. His research interests include cloud computing, spatial data science, and IoT.